

Föreläsning 19

Ändrad
7 Oct
11:08

En *fil* är en sekvens tecken, eller följd av tecken. Ni har lagrat era labbar på olika filer, och era labbrapporter.

Man kan läsa från filer i stället för från tangentbordet. Och skriva på filer i stället för på bildskärmen.

Ni arbetar mot filer likadant som ni arbetat mot tangentbord och bildskärm. Skillnaden är att funktionerna inte heter likadant.

Här visas släktskapet mellan gamla och nya funktioner:

stdin/stdout	fil
<code>scanf(format,argument...)</code>	<code>fscanf(fil,format,argument...)</code>
<code>printf(format,argument...)</code>	<code>fprintf(fil,format,argument...)</code>
<code>getchar()</code>	<code>fgetc(fil)</code>
<code>putchar(tecken)</code>	<code>fputc(tecken,fil)</code>
<code>gets(sträng)</code>	<code>fgets(sträng,maxlängd,fil)</code>
<code>puts(sträng)</code>	<code>fputs(sträng,fil)</code>

Dessutom behöver vi två nya funktioner:

■ Funktionen `fopen()` bestämmer vilken fil programmet ska utnyttja. Om er programmeringsmiljö *Borland C* är skriven i C, så anropas `fopen()` när ni ger kommando *Open* eller *New*.

■ Funktionen `close()` stänger filen. Det motsvarar kommandot *Close* i *Borland C*.

Filer har typen `FILE`. Eller rättare sagt: `FILE`-pekare. Vi har redan stött på två filer av denna typ: `stdin` och `stdout`.

Så nära är släktskapet till filhantering, att följande är synonymt:

stdin/stdout	fil
<code>scanf("%c", &ch)</code>	<code>fscanf(stdin, "%c", &ch)</code>
<code>printf("%c", ch)</code>	<code>fprintf(stdout, "%c", ch)</code>
<code>getchar()</code>	<code>fgetc(stdin)</code>
<code>putchar(ch)</code>	<code>fputc(ch, stdout)</code>
<code>gets(s)</code>	<code>fgets(s, max, stdin)(**)</code>
<code>puts(s)</code>	<code>fputs(s, stdout)(**)</code>

FOTNOTER

(*) `fgets()` tar en extra parameter `max` som anger hur många tecken som högst ska läsas in. Oavsett hur lång raden är.

Den parametern borde `gets()` också ha! Den kan man använda för att gardera sig mot att rader inte får plats i `s`.

(**) `puts(s)` skriver ut `s` plus ett nyradstecken. `fputs(s, stdout)` skriver däremot *inte* ut ett extra nyradstecken. Men det gör ingenting! För `fgets()` *läser in* nyradstecknet, men det gör inte `gets()`.

Parvis fungerar alltså funktionerna tillsammans på samma sätt. Följande två rader fungerar likadant -- kopierar en rad från `stdin` till `stdout`:

```
puts(gets(s));
fputs(fgets(s, max, stdin), stdout);
```

En del av filfunktionerna lägger filargumentet först, medan andra lägger det sist.

Jag har ingen aning om vad denna memoreringssaboterande inkonsekvens beror på. Som vanligt är det inte säkert att kompilatorn varnar.

Sök-byt igen

Alla exempelprogram ni sett hittills kan ganska enkelt anpassas till att läsa från filer och skriva på filer -- istället för tangentbord och bildskärm.

Som exempel tar jag *Sök-byt ut*. Det är bara huvudprogrammet som behöver ändras. Jag har markerat det som är nytt eller ändrat:

```
17 void main()
18 {
19     char w1[MAXWORD], w2[MAXWORD];
20     char s[MAXLINE];
21     char *p;
22     int length_w1;
23     FILE *input, *output;
24
25     printf("Läs från fil: ");
26     gets(s);
27     input = fopen(s, "r");
28
29     printf("Skriv på fil: ");
30     gets(s);
31     output = fopen(s, "w");
32
33     printf("Ändra från: ");
34     gets(w1);
35     printf("Till: ");
36     gets(w2);
37
38     length_w1 = strlen(w1);
39
40     while (fgets(s, MAXLINE, input) != NULL) {
41         p = s;
42         while (*p != '\0')
43             if (prefix(w1, p)) {
44                 fprintf(output, "%s", w2);
45                 p = p + length_w1;
46             } else
```

```

47             fputc(*p++, output);
48         /* fputc('\n', output); */
49     }
50     fclose(input);
51     fclose(output);
52 }

```

- Rad 23 deklarerar in- och utfil, `input`, och `output`.
- Rad 26 läser in namnet på infilen till strängen `s`.
- Rad 27 säger:
 - att vi ska *läsa* från `input` ("`r`" = "read")
 - att `input` motsvaras av filen med det fysiska namnet `s`.

Man säger att vi "öppnar `input` för läsning".

Filen heter `s` där den ligger lagrad på hårdisken. `s` är filens *fysiska namn* -- `input` är filens *logiska namn*.

- Rad 31 öppnar `output` för skrivning ("`w`" = "write").
- Rad 40 läser in en rad från `input` till `s`. Dock högst `MAXLINE` tecken. Om fler än `MAXLINE` tecken finns på raden lämnas dessa kvar -- olästa -- de blir lästa i nästa anrop.
- Rad 44 skriver `w2` på `output`.
- Rad 47 skriver ett tecken på `output`.
- Rad 48 är bortkommenterad eftersom `fgets()` har läst in ett nyradstecken -- och det skrivs ut på rad 47.
- Rad 50 och 51 stänger `input` och `output`.

Övningsuppgift

Vad händer om en rad är längre än `MAXLINE`? Klipper programmet upp den raden i två rader?

Nästa lektion

